INDIVIDUAL GENERATIVE AI ASSIGNMENT

Implementation of a Retrieval-Augmented Generation (RAG) System

Research-paper-based implementation report

Student Name	Tanishka Vishnu Tapkir
PRN	202401110071
Course Code	2311312T
Department / Branch	Artificial Intelligence and Machine Learning
Institution	MIT Academy of Engineering

Academic Year: 2026–2027

1. Abstract

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with a generative language model. Instead of depending only on information stored in the model parameters, a RAG system first retrieves relevant content from an external knowledge source and then uses that content to prepare an answer. This report describes an adapted implementation of a document-based question-answering system inspired by the research paper *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* by Patrick Lewis and co-authors (2020).

The implemented application accepts a PDF document, extracts its text, divides the text into overlapping chunks, converts the chunks into vector embeddings, and stores the vectors in a FAISS index. When a user asks a question, the system embeds the question and retrieves the three most similar chunks. The retrieved text is then passed as context to a Groq-hosted language model, which generates a response. Source page numbers are retained to make the retrieved information easier to inspect.

This implementation demonstrates the core RAG pipeline on an uploaded PDF. It is an educational adaptation and not a full reproduction of the original research system, which used a Wikipedia-based dense retrieval index and pretrained sequence-to-sequence models.

2. Aim

To study the core methodology of Retrieval-Augmented Generation and implement a PDF-based question-answering system that retrieves relevant document passages and uses a generative language model to answer user questions.

3. Objectives

- Study the architecture and working principle of the selected RAG research paper.
- Extract text from an uploaded PDF and divide it into manageable chunks.
- Generate semantic embeddings using the all-MiniLM-L6-v2 sentence-transformer model.
- Create a FAISS vector index and retrieve the top three relevant chunks for a query.
- Use a Groq language model to generate answers from retrieved context.
- Display source page references and evaluate the implementation using test questions and retrieval-time measurements.

4. Research Paper Overview

Selected paper: Patrick Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems (NeurIPS), 2020. The paper introduced a framework that combines a neural retriever with a sequence-to-sequence generator. The retriever finds relevant passages from a large external knowledge collection, while the generator uses the retrieved passages to produce output.

The central idea is to make generation depend on both the input query and retrieved non-parametric knowledge. This can improve performance on tasks that require factual or knowledge-intensive information and makes it possible to update the external knowledge source without retraining the entire language model.

5. System Architecture

The implemented system follows a simplified RAG workflow. The PDF acts as the external knowledge source. Its text is processed into chunks, and each chunk is represented by a dense vector. FAISS stores these vectors and performs similarity search. The retrieved passages are supplied to the Groq language model as context.

Stage	Component / Tool	Purpose
1	PDF input + PyPDF	Upload the document and extract page-wise text.
2	Chunking	Split text into 500-word chunks with 100-word overlap.
3	Sentence Transformers	Convert chunks into dense vector embeddings.
4	FAISS IndexFlatIP	Store vectors and search for similar passages.
5	Retriever	Return the top three matching chunks for a question.
6	Groq LLM API	Generate an answer using the retrieved context.
7	Source references	Show the page numbers of retrieved passages.

Workflow: PDF Upload → Text Extraction → Chunking → Embeddings → FAISS Vector Index → Similarity Retrieval → Context Construction → Groq LLM → Answer with Source Pages

6. Tools and Technologies

- **Python:** Main programming language used for the implementation.
- **Google Colab:** Notebook environment used to execute and document the implementation.
- **PyPDF:** Extracts text from PDF pages.
- **Sentence Transformers (all-MiniLM-L6-v2):** Generates compact semantic embeddings.
- **FAISS:** Performs efficient vector similarity search.
- **Groq API (openai/gpt-oss-20b):** Generates natural-language answers from the retrieved context.
- **NumPy and Pandas:** Support numerical operations and evaluation-result handling.

7. Methodology

The implementation was completed as a sequence of processing stages. First, the PDF was uploaded and its text was extracted page by page. The page number was retained with each text segment so that source references could be displayed later.

The extracted text was split into chunks of 500 words. A 100-word overlap was used between consecutive chunks to reduce the chance of losing context at chunk boundaries. Each chunk was passed through all-MiniLM-L6-v2 to produce a normalized vector embedding. The vectors were converted to float32 and added to a FAISS IndexFlatIP index. Since the vectors were normalized, inner-product search acts as cosine-similarity search.

7. Methodology (continued)

For a user question, the same embedding model converts the question into a vector. FAISS searches the index and returns the three highest-scoring chunks. The retrieved chunks, including their page numbers, are assembled into a context prompt. The Groq API is then called with instructions to answer using only the supplied context and to indicate when the answer is not available in the document. This instruction is intended to reduce unsupported answers, although it cannot guarantee that hallucinations will never occur.

The notebook also runs a small set of test questions. For each question, it records the generated answer and the page numbers of the retrieved chunks. Retrieval time is measured separately. Retrieval time is not the same as answer accuracy; answer quality must be assessed by comparing the generated answer with the source document.

8. Implementation Details

Parameter	Selected setting	Reason / role
Chunk size	500 words	Creates passages small enough for focused retrieval.
Chunk overlap	100 words	Preserves some context across adjacent chunks.
Embedding model	all-MiniLM-L6-v2	Produces semantic vector representations.
Vector index	FAISS IndexFlatIP	Performs exact inner-product search.
Retrieved passages	Top 3	Limits the context to the most similar chunks.
Generation model	openai/gpt-oss-20b via Groq	Produces a natural-language answer from context.

9. Results and Observations

The implemented pipeline is designed to demonstrate the following observable outputs:

- The PDF text extraction stage displays the number of pages successfully extracted and sample extracted text.
- The chunking stage displays the number of generated chunks and a sample chunk.
- The embedding stage reports the embedding array shape.
- The FAISS stage reports the number of vectors stored in the index.
- For a question, the retrieval stage displays the selected passages, similarity scores, and source page numbers.
- The generation stage displays a response produced by the Groq language model.
- The evaluation stage records answers and retrieved page references for a set of test questions and calculates average retrieval time.

Insert actual output evidence before submission: Add screenshots from your own Colab execution showing (1) successful PDF extraction, (2) chunks and embeddings, (3) FAISS retrieval with scores and page numbers, (4) a generated answer, and (5) the evaluation table and measured retrieval time. Do not enter estimated or invented measurements.

10. Evaluation and Analysis

The notebook uses five example questions to check different parts of the sample knowledge source: the meaning of RAG, the purpose of chunking, the role of embeddings, the use of FAISS, and common limitations of RAG. For each question, the system retrieves up to three chunks and generates an answer from those chunks.

The evaluation should be reviewed manually. A useful answer should address the question, be supported by the retrieved text, and avoid adding facts that are not present in the source. The page references provide a way to inspect the evidence. Similarity scores indicate how closely a question embedding matches a chunk embedding; they should not be interpreted as answer accuracy or probability of correctness.

The average retrieval time is calculated by timing the retrieval function over the test questions. This value measures the time used by the local embedding and FAISS retrieval stages in the current notebook environment. It does not include the time required for the Groq response unless generation is timed separately. Results may vary depending on the document size, runtime hardware, and model loading state.

Actual evaluation values to enter from the executed notebook:

Measure	Actual value from execution
Number of PDF pages extracted	Enter the value printed by the notebook
Number of text chunks	Enter the value printed by the notebook
Embedding dimensions	Enter the embedding shape
Number of vectors in FAISS	Enter the index.ntotal value
Number of test questions	5 (if all five questions were run)
Average retrieval time	Enter the measured seconds from your output

11. Limitations

- The current notebook processes one uploaded PDF at a time and does not provide a persistent multi-document knowledge base.
- Text extraction quality depends on the PDF. Scanned pages may require OCR, which is not included in this version.
- A fixed chunk size and overlap may not be ideal for every document structure.
- The system retrieves only three passages; relevant information may be missed if it is spread across more chunks.
- The generator may still produce incorrect or unsupported statements even when prompted to use the context.
- The Groq API requires internet access, an available model, and a valid API key. API usage may be subject to rate limits.

12. Future Scope

- Add support for multiple PDFs and maintain a reusable document collection.
- Use OCR to process scanned documents and image-based PDF pages.
- Experiment with semantic or structure-aware chunking rather than relying only on a fixed word count.
- Compare different embedding models and evaluate their effect on retrieval relevance.
- Add a reranking stage to reorder retrieved passages using a stronger relevance model.
- Measure answer quality with human evaluation or suitable question-answering metrics, in addition to retrieval time.
- Add persistent chat history, document summaries, and a user-friendly interface.
- Explore access control and secure document storage for multi-user use.

13. Conclusion

This assignment studied the core idea of Retrieval-Augmented Generation and implemented an adapted PDF question-answering pipeline. The system extracts text from a PDF, divides it into overlapping chunks, converts the chunks into embeddings, stores the vectors in FAISS, retrieves relevant passages for a question, and uses a Groq-hosted language model to generate an answer based on the retrieved context.

The implementation provides practical experience with document processing, semantic embeddings, vector search, and context-grounded generation. It also demonstrates the importance of retaining source references and evaluating retrieval and generation separately. The work is based on the methodology of the selected research paper, but it is a simplified educational implementation rather than a reproduction of the original experimental system.

14. References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. Available: https://arxiv.org/abs/2005.11401

[2] Sentence Transformers documentation. Available: https://www.sbert.net/

[3] FAISS documentation. Available: https://faiss.ai/

[4] PyPDF documentation. Available: https://pypdf.readthedocs.io/

[5] Groq API documentation. Available: https://console.groq.com/docs

Appendix: Evidence Screenshots

Attach your own screenshots from Google Colab after executing the notebook. Recommended evidence: PDF upload and extraction; chunk count and sample chunk; embedding shape and FAISS vector count; retrieved passages with similarity scores and page references; generated answer; evaluation table and measured retrieval time.